Hochschule Konstanz
Fakultät Informatik

Software Engineering

07 - Design Pattern I

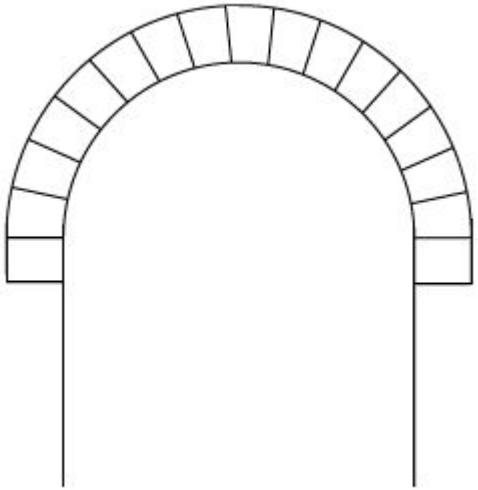
Prof. Dr. Marko Boger

What are Design Pattern?

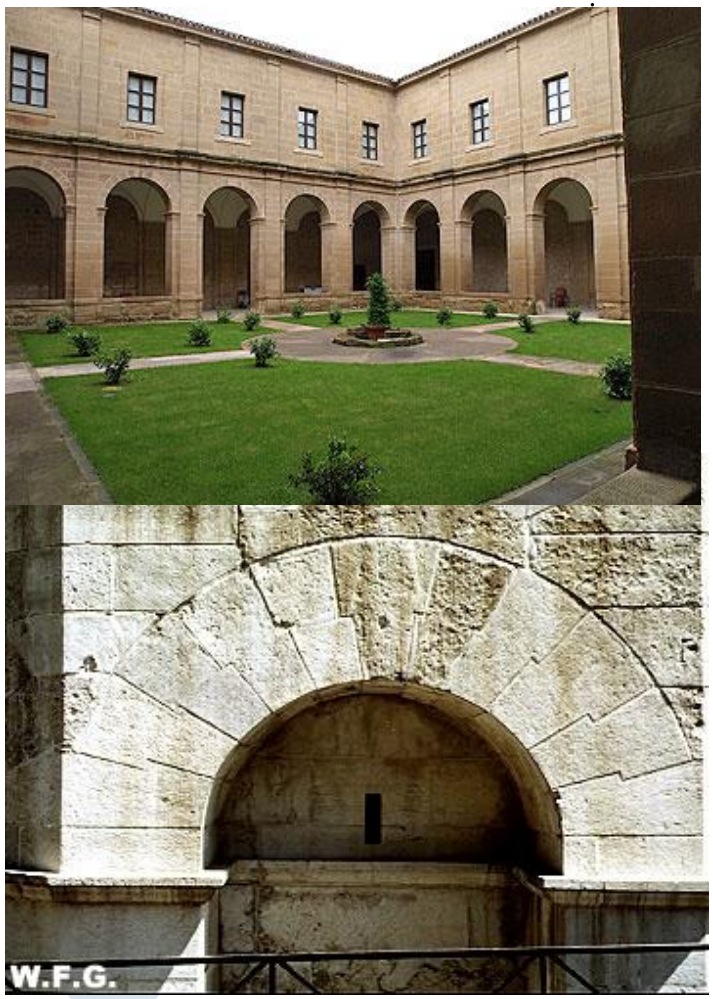
A design pattern is a transferable solution to a problem.

Like the concept of an arch.

It is an idea, a concept, that needs to be adopted to the situation.



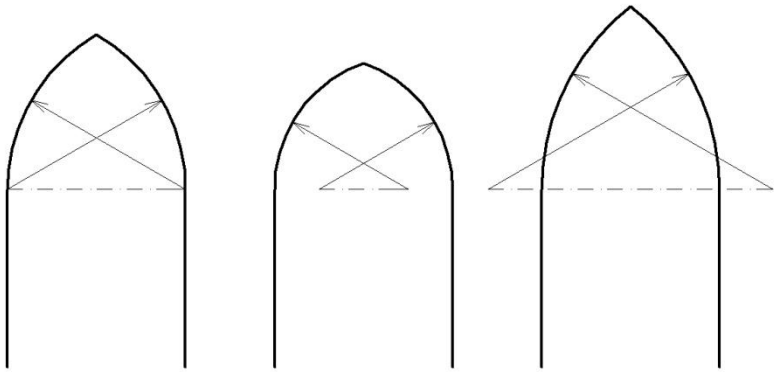
Romanesque Round Arch



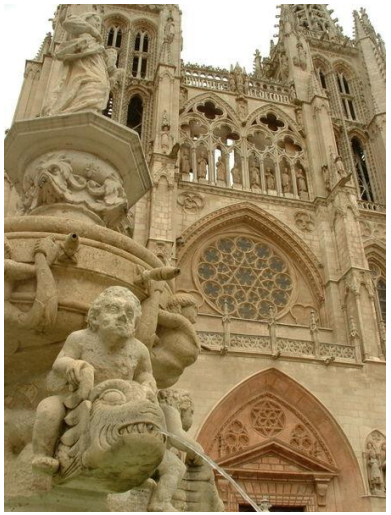
Design Pattern

Design patterns appear over and over again. Different pattern have different qualities or provide solutions to different problems.

A pointed arch allowed higher buildings and is the characteristic design element of Gothic Architecture



Gothic Pointed Arches



Design Pattern in IT

- Design Patterns were first described by Christopher Alexander in the field of civil Architecture
- The concept was then transferred to Software Engineering
- In software engineering, a design pattern is a general reusable solution to a commonly occurring problem
- Design Patterns capture and communicate proven design ideas
- Design Patterns describe the problem, and its solution in a structured and abstract way, so that it can be transferred to other situations
- A design pattern is not a finished building block that can be applied directly. It is a description or template for how to solve a problem.
- They have become part of our domain language and help to find and communicate design solutions

Erich Gamma

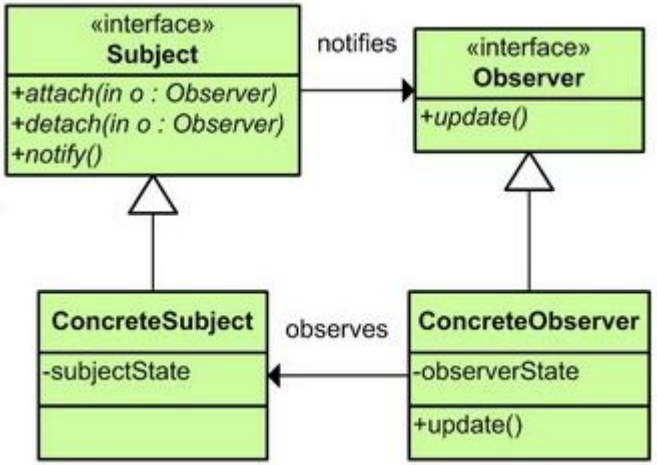
- PhD on Design Patterns
- Main-Author of the book
  Design Pattern
- Introduced the term design pattern into the domain software engineering
- Co-Author of JUnit

- Head of the Eclipse-Project
- Head of the Jazz-Project
- Lead Dev on Visual Studio Code
- Office 365

Observer Pattern

The Observer Pattern is a solution to the problem, how to invert a dependency.

```
trait Observer {
 def update: Boolean
}
class Observable {
 var subscribers: Vector[Observer] = Vector()
 def add(s: Observer): Unit = subscribers = subscribers
:+ s
 def remove(s: Observer): Unit = subscribers =
subscribers.filterNot(o => o == s)
 def notifyObservers: Unit = subscribers.foreach(o =>
o.update)
}
```



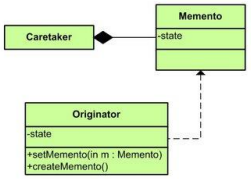
Pattern Cheat Sheet

C Abstract Factory	S Facade	S Proxy
S Adapter	C Factory Method	B Observer
S Bridge	S Flyweight	C Singleton
C Builder	B Interpreter	B State
B Chain of Responsibility	B Iterator	B Strategy
B Command	B Mediator	B Template Method
S Composite	B Memento	B Visitor
S Decorator	C Prototype	

Memento

Type: Behavioral

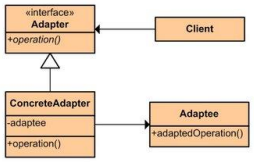
What it is:
Without violating encapsulation, capture and externalize an object's internal state so that the object can be restored to this state later.



Adapter

Type: Structural

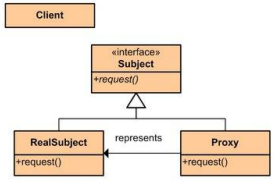
What it is:
Convert the interface of a class into another interface clients expect. Lets classes work together that couldn't otherwise because of incompatible interfaces.



Proxy

Type: Structural

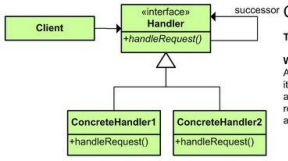
What it is:
Provide a surrogate or placeholder for another object to control access to it.



Chain of Responsibility

Type: Behavioral

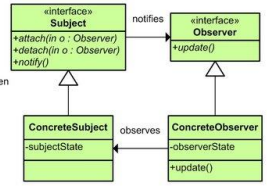
What it is:
Avoid coupling the sender of a request to its receiver by giving more than one object a chance to handle the request. Chain the receiving objects and pass the request along the chain until an object handles it.



Observer

Type: Behavioral

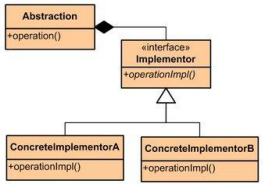
What it is:
Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.



Bridge

Type: Structural

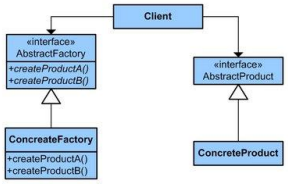
What it is:
Decouple an abstraction from its implementation so that the two can vary independently.



Abstract Factory

Type: Creational

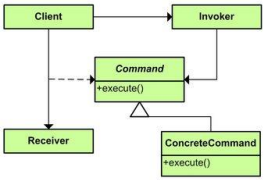
What it is:
Provides an interface for creating families of related or dependent objects without specifying their concrete class.



Command

Type: Behavioral

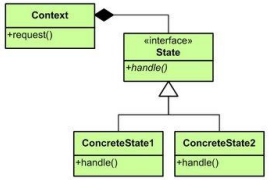
What it is:
Encapsulate a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations.



State

Type: Behavioral

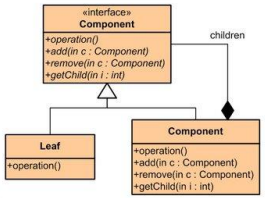
What it is:
Allow an object to alter its behavior when its internal state changes. The object will appear to change its class.



Composite

Type: Structural

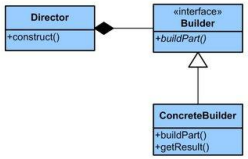
What it is:
Compose objects into tree structures to represent part-whole hierarchies. Lets clients treat individual objects and compositions of objects uniformly.



Builder

Type: Creational

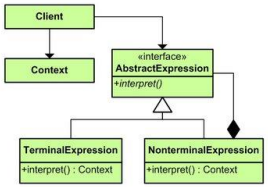
What it is:
Separate the construction of a complex object from its representing so that the same construction process can create different representations.



Interpreter

Type: Behavioral

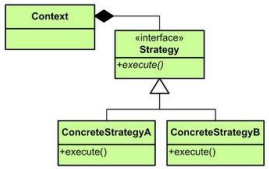
What it is:
Given a language, define a representation for its grammar along with an interpreter that uses the representation to interpret sentences in the language.



Strategy

Type: Behavioral

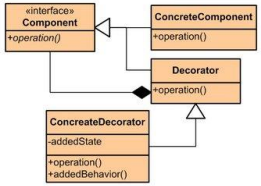
What it is:
Define a family of algorithms, encapsulate each one, and make them interchangeable. Lets the algorithm vary independently from clients that use it.



Decorator

Type: Structural

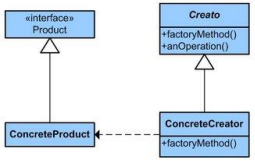
What it is:
Attach additional responsibilities to an object dynamically. Provide a flexible alternative to sub-classing for extending functionality.



Factory Method

Type: Creational

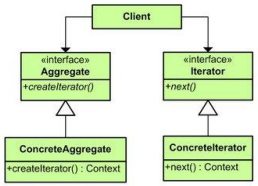
What it is:
Define an interface for creating an object, but let subclasses decide which class to instantiate. Lets a class defer instantiation to subclasses.



Iterator

Type: Behavioral

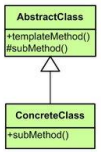
What it is:
Provide a way to access the elements of an aggregate object sequentially without exposing its underlying representation.



Template Method

Type: Behavioral

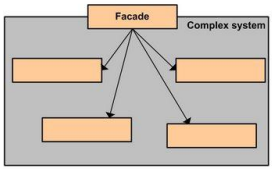
What it is:
Define the skeleton of an algorithm in an operation, deferring some steps to subclasses. Lets subclasses redefine certain steps of an algorithm without changing the algorithm's structure.



Facade

Type: Structural

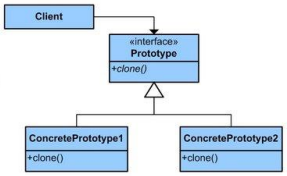
What it is:
Provide a unified interface to a set of interfaces in a subsystem. Defines a high-level interface that makes the subsystem easier to use.



Prototype

Type: Creational

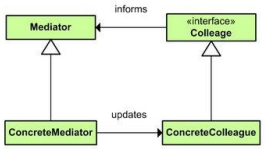
What it is:
Specify the kinds of objects to create using a prototypical instance, and create new objects by copying this prototype.



Mediator

Type: Behavioral

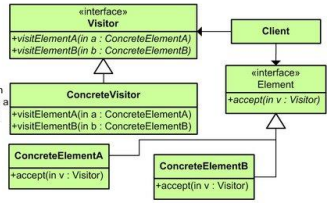
What it is:
Define an object that encapsulates how a set of objects interact. Promotes loose coupling by keeping objects from referring to each other explicitly and it lets you vary their interactions independently.



Visitor

Type: Behavioral

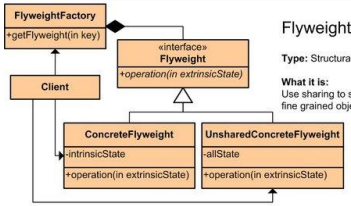
What it is:
Represent an operation to be performed on the elements of an object structure. Lets you define a new operation without changing the classes of the elements on which it operates.



Flyweight

Type: Structural

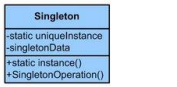
What it is:
Use sharing to support large numbers of fine grained objects efficiently.



Singleton

Type: Creational

What it is:
Ensure a class only has one instance and provide a global point of access to it.



Copyright © 2007 Jason S. McDonald
http://McDonaldLand.wordpress.com

Gamma, Erich; Helm, Richard; Johnson, Ralph; Vlissides, John (1995). *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, Massachusetts: Addison Wesley Longman, Inc.

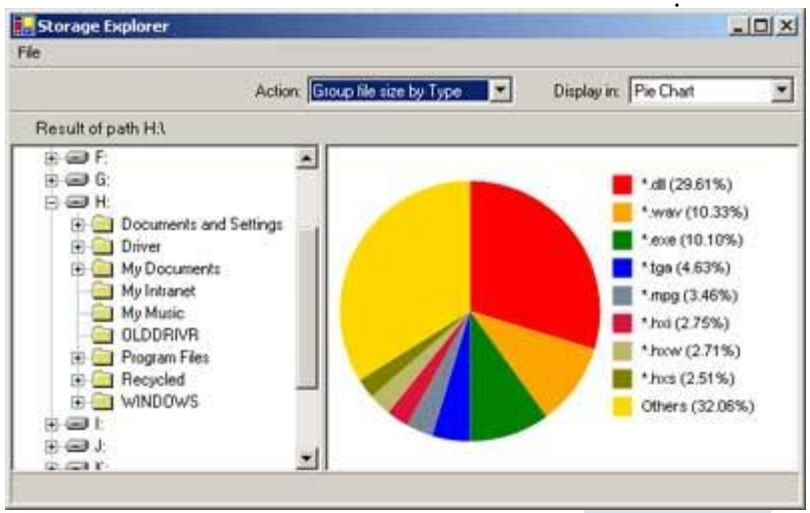
Copyright © 2007 Jason S. McDonald
http://McDonaldLand.wordpress.com

Gamma, Erich; Helm, Richard; Johnson, Ralph; Vlissides, John (1995). *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, Massachusetts: Addison Wesley Longman, Inc.

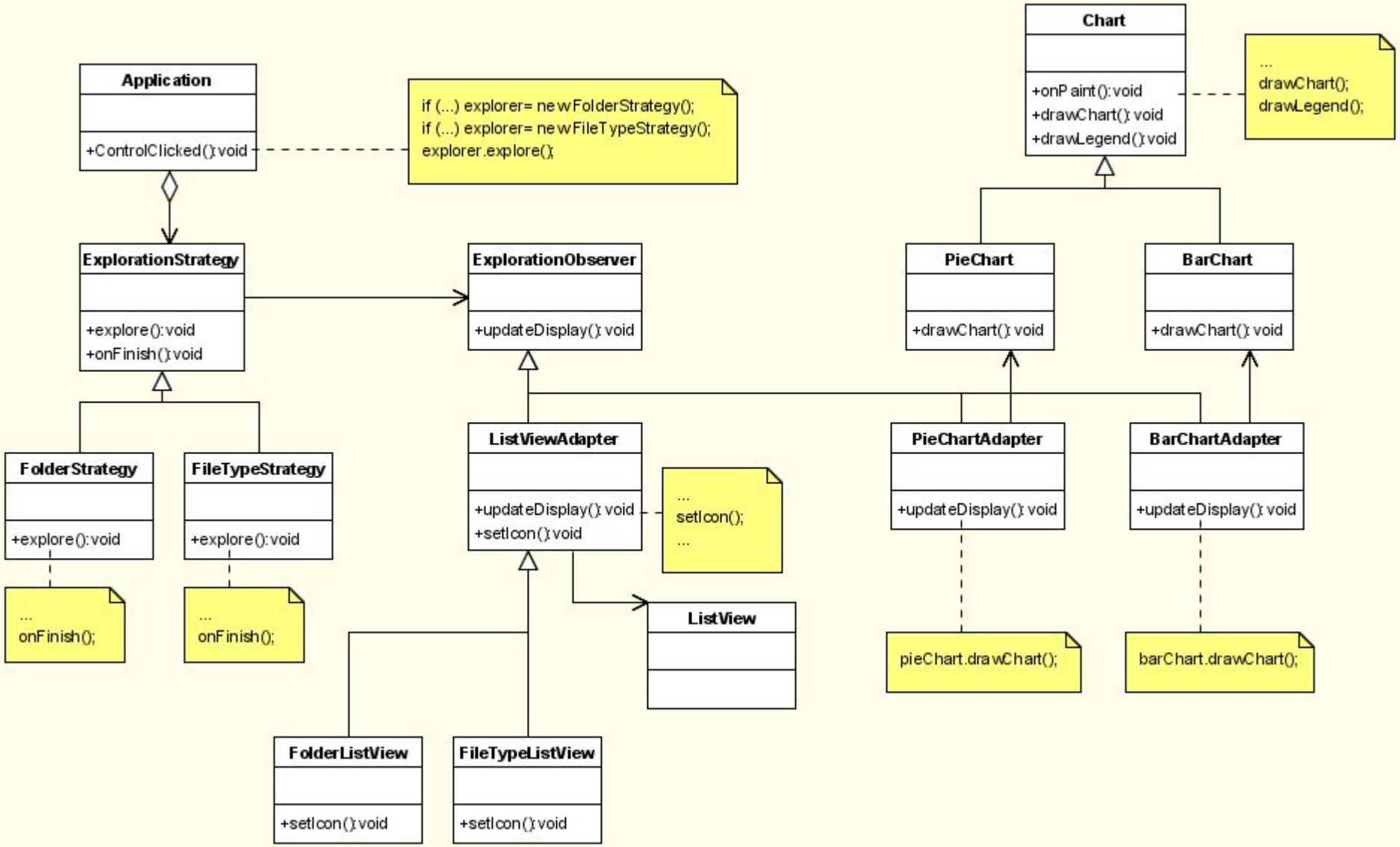
Example for a Pattern-based Design

A Storage Explorer

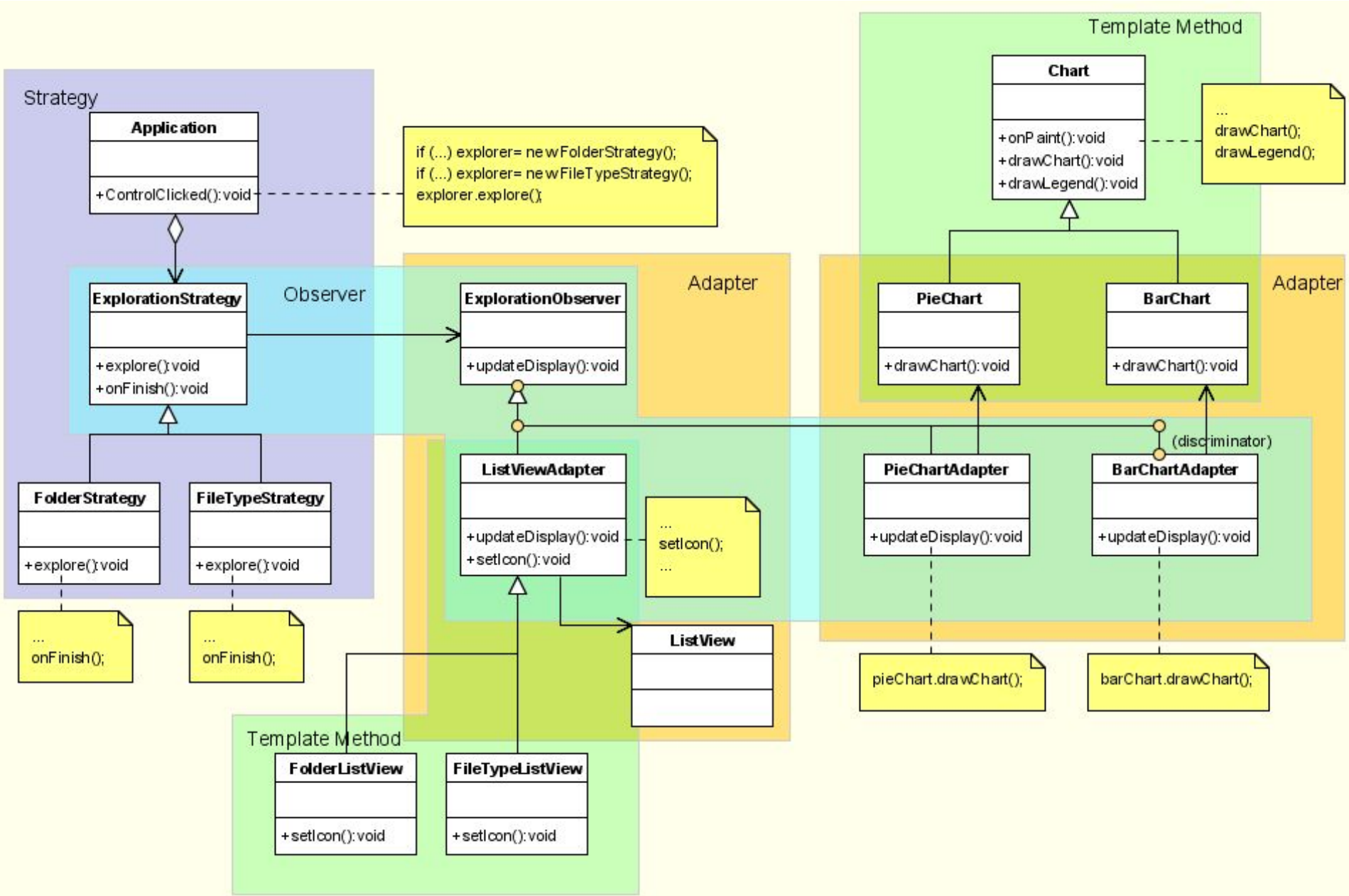
- Analyzes File Size
- Tree View of the File Structure
- Percentage or Size of Files/Folders
  - Pie
  - Chart
  - List
- View by
  - Filetype
  - Folderstructure



Structure of the Storage Explorer



Structure of the Storage Explorer



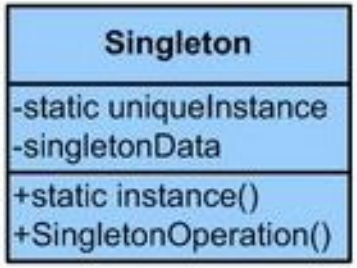
Singleton

Singleton is a pattern to ensure there is only one instance of a class. If the singleton contains variable data, it should only be used if exactly one instance has to be enforced.

Singleton

Type: Creational

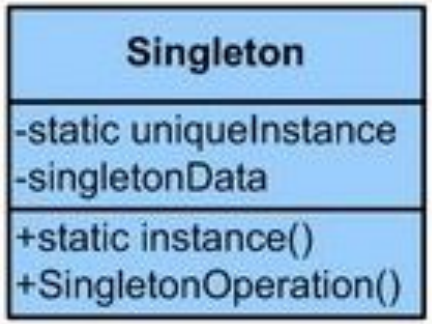
What it is:
Ensure a class only has one instance and provide a global point of access to it.



Singleton Java-style

Singleton has a private instance (and a private constructor)

```
class Singleton1 {
 def singletonFunction = println("I am a singleton")
}
object Singleton1 {
 private var instance:Singleton1 = null
 def getInstance:Singleton1= {
   if (instance == null) {
     instance= new Singleton1()
   }
   instance
 }
}
Singleton1.getInstance.singletonFunction
```



Singleton Scala-style

- Scala has Singletons built in: Object

```
object Singleton {
 def singletonFunction = println("I am a singleton")
}

Singleton.singletonFunction
```

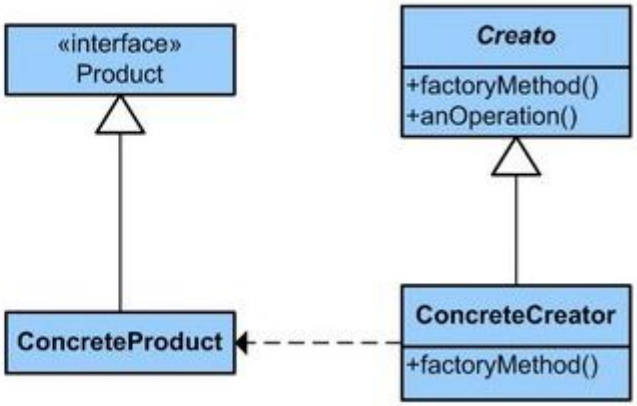
Factory Method

A Factory Method is used to create an instance of an abstraction.
The dependency to the concrete type is avoided.

Factory Method

Type: Creational

What it is:
Define an interface for creating an object, but let subclasses decide which class to instantiate. Lets a class defer instantiation to subclasses.



FactoryMethod Java-style

Get a specific instance of an abstraction from a factory

```
trait Animal1 {
 def run = println("animal running")
}
private class Dog1 extends Animal {
 override def run: Unit = println("dog running")
}
private class Cat1 extends Animal {
 override def run: Unit = println("cat running")
}
object AnimalFactory {
 def getInstance(kind: String) = kind match {
   case "dog" => new Dog1()
   case "cat" => new Cat1()
 }
}
val dog1 = AnimalFactory.getInstance("dog")
dog1.run
```

FactoryMethod Scala-style

Scala has FactoryMethod built in: apply on companion object

```
trait Animal {
 def run = println("animal running")
}
private class Dog extends Animal {
 override def run: Unit = println("dog running")
}
private class Cat extends Animal {
 override def run: Unit = println("cat running")
}
object Animal {
 def apply(kind: String) = kind match {
   case "dog" => new Dog()
   case "cat" => new Cat()
 }
}
val animal = Animal("dog")
animal.run
```

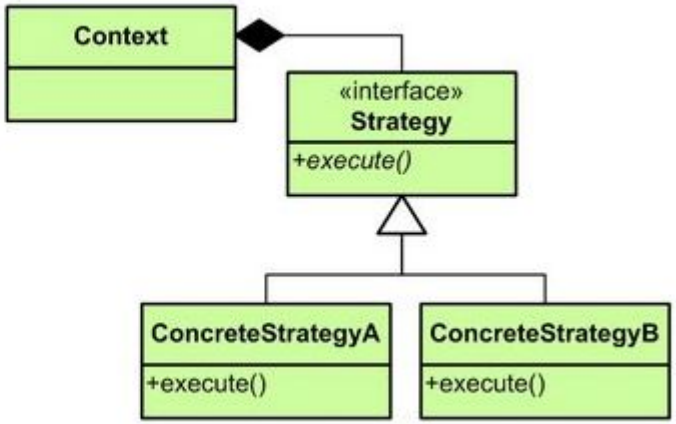
Strategy Pattern

A Strategy allows to switch between different algorithms.

Strategy

Type: Behavioral

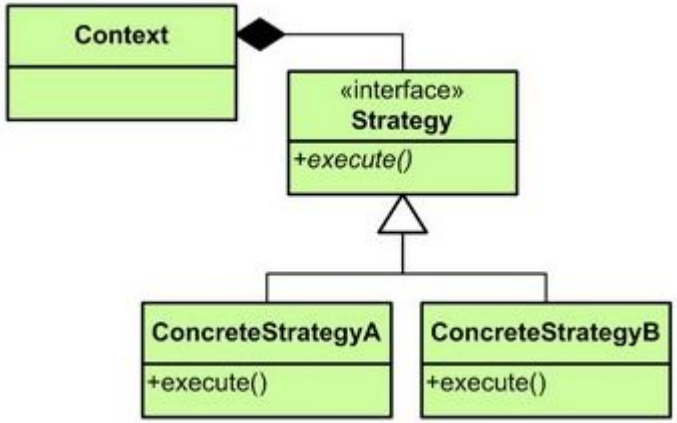
What it is:
Define a family of algorithms, encapsulate each one, and make them interchangeable. Lets the algorithm vary independently from clients that use it.



Strategy Java-style

Strategy with Inheritance

```
object Context1 {
 trait Strategy {
   def execute
 }
 class Strategy1 extends Strategy {
   override def execute = println("I am strategy 1")
 }
 class Strategy2 extends Strategy {
   override def execute = println("I am strategy 2")
 }
 var strategy = if (Random.nextInt() % 2 == 0) new Strategy1 else new Strategy2
}

Context1.strategy.execute
```



Strategy Scala-style

Scala allows to override the function value at runtime

```
object Context2 {

 var strategy = if (Random.nextInt() % 2 == 0) strategy1 else strategy2

 def strategy1 = println("I am strategy 1")

 def strategy2 = println("I am strategy 2")
}

Context2.strategy
```

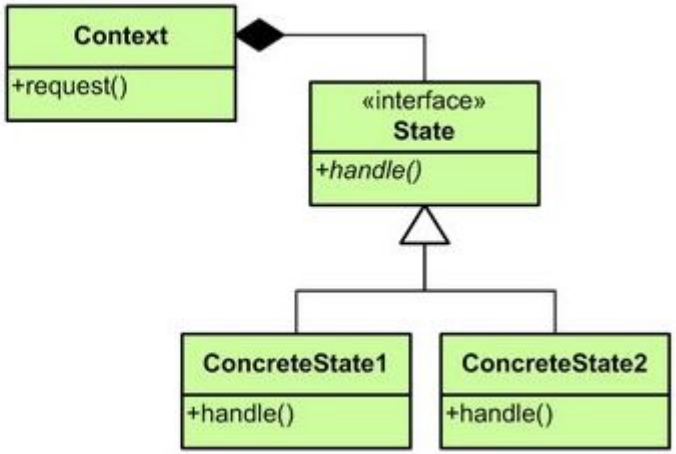
State Pattern

Some context code before we look at the implementation

State

Type: Behavioral

What it is:
Allow an object to alter its behavior when its internal state changes. The object will appear to change its class.

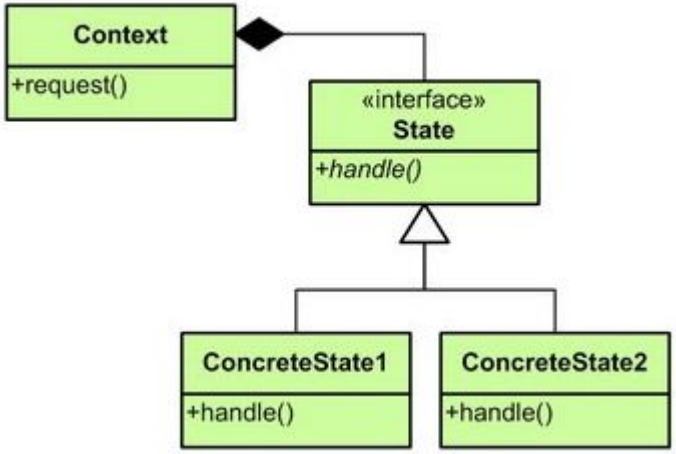



```
trait Event

case class OnEvent() extends Event

case class OffEvent() extends Event

StateContext1.handle(new OnEvent)
StateContext1.handle(new OffEvent)

StateContext2.handle(OnEvent())
StateContext2.handle(OffEvent())
```

State Java-style

In Java the State is encapsulated in Instances of an Abstraction

```
object StateContext1 {
 trait State {
   def handle(e: Event): State
 }
 case class OnState() extends State {
   println("I am On")
   override def handle(e: Event): State = {
     e match {
       case on: OnEvent => OnState()
       case off: OffEvent => OffState()
     }
   }
 }
 case class OffState() extends State {
   println("I am Off")
   override def handle(e: Event): State = {
     e match {
       case on: OnEvent => OnState()
       case off: OffEvent => OffState()
     }
   }
 }
 var state: State = OffState()
 def handle(e: Event) = state = state.handle(e)
}
```



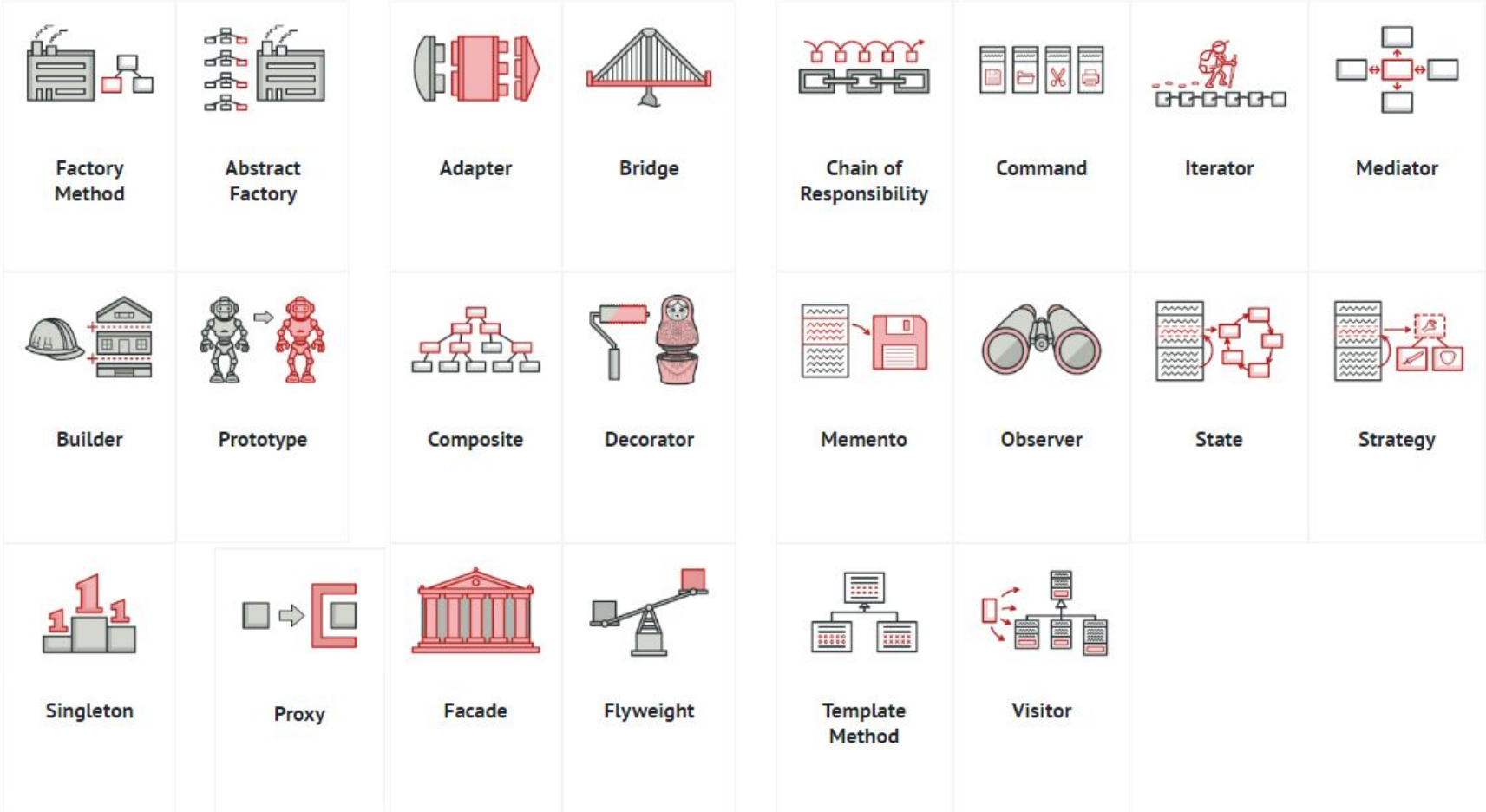
State Scala-style

In Scala this can be encapsulated in a reassignable function.

```
object StateContext2 {
 var state = onState
 def handle(e: Event) = {
   e match {
     case on: OnEvent => state = onState
     case off: OffEvent => state = offState
   }
   state
 }

 def onState = println("I am on")
 def offState = println("I am off")
}
```

Resources

Refactoring Guru: https://refactoring.guru/design-patterns



Design Principles

DRY - Don‘t Repeat yourself

KISS - Keep it simple, Stupid

YAGNI - You Ain't Gonna Need It

SOLID

First Broken Window Principle

Law of Demeter

Law of least astonishment

Single Resposibility Principle

A class should have only a single responsibility (i.e. only one potential change in the software's specification should be able to affect the specification of the class)

Open / Closed Principle

A software module (it can be a class or method) should be open for extension but closed for modification.

Liskov Substitution Principle

Objects in a program should be replaceable with instances of their subtypes without altering the correctness of that program.

Interface Segregation Principle

Clients should not be forced to depend upon the interfaces that they do not use.

Dependency Inversion Principle

Program to an interface, not to an implementation.

Task 7: Integrate Patterns into your code

Increase the quality of your code by making it more extendable using patterns.

Try to integrate a pattern like Strategy, Factory, State, Template or others.

Try to implement 3 or more patterns.